KARADENIZ TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF SOFTWARE ENGINEERING

MINI LANGUAGE PARSER AND PARSE TREE GENERATOR

Student Name: HUSNA AYDIN - NISA GUCER
Student Number: 439198 - 439211

Date: May 4, 2026

Contents

1 Introduction **2**

2 Aims and Objectives **2**

3 Methodology **3**

3.1 Reading the Grammar . . . 3
3.2 Non-terminal and Terminal Detection . . . 3
3.3 Recursive Descent Parsing . . . 3
3.4 Backtracking . . . 4
3.5 Handling Epsilon Productions . . . 4
3.6 Tokenization Strategy . . . 4
3.7 Parse Tree Construction . . . 4
3.8 JSON Conversion . . . 5
3.9 Error Handling . . . 5

4 Findings **5**

4.1 Grammar 1: Word-Based Grammar . . . 5
4.1.1 Example Parse Tree Output . . . 6
4.1.2 Example JSON Output . . . 6
4.1.3 Example Error Output . . . 7
4.2 Grammar 2: Character-Based Grammar . . . 7
4.2.1 Example Parse Tree Output . . . 8
4.2.2 Example JSON Output . . . 8
4.2.3 Epsilon Example . . . 9

5 Difficulties Encountered During Implementation **9**

6 Conclusion and Discussion **10**

7 Recommendations **10**

Abstract

In this project, a parser was developed to analyze sentences according to grammar rules written in Backus-Naur Form (BNF). The program reads grammar rules from a file, checks whether each input sentence is syntactically valid, and generates a parse tree and JSON representation for valid inputs. For invalid inputs, the system reports the error location, the expected symbol, and the reason for invalidity. The parser was implemented using a recursive descent approach with backtracking. In addition, epsilon productions were handled as special cases, and the tokenizer was designed to support both word-based and character-based grammars. The implementation was tested with two different grammars. The first grammar was based on English-like word sequences, while the second grammar used recursive character-level rules. Test results showed that the parser correctly identified valid and invalid inputs, produced readable parse trees, and generated meaningful JSON outputs and error messages.

Keywords

BNF, recursive descent parser, parse tree, JSON, syntax analysis

1 Introduction

Formal grammars are widely used in programming languages, compilers, interpreters, and syntax analysis systems. A grammar defines the valid structure of a language and determines how symbols can be combined. In compiler design, parsing is the process of checking whether an input sequence follows the given grammar rules.

The purpose of this project is to implement a parser that reads a grammar written in BNF format and tests whether given input sentences conform to the grammar. If the input is valid, the parser produces both a parse tree and a JSON representation. If the input is invalid, the program reports where the error occurs and what was expected at that point.

This project is important because it provides a practical implementation of context-free grammar parsing and helps in understanding recursive structures, syntax rules, parse tree generation, and error handling.

2 Aims and Objectives

The main aim of this project is to design and implement a parser for BNF-based grammars.

The objectives of the project are as follows:

- To read grammar rules from a text file and store them in an appropriate data structure.
- To analyze input sentences according to the given grammar rules.
- To determine whether each sentence is valid or invalid.
- To generate a parse tree for valid inputs.
- To convert the parse tree into JSON format.

- To detect syntax errors and produce meaningful error messages.
- To support different grammar styles, including both word-based and character-based grammars.
- To handle epsilon productions correctly.

3 Methodology

The parser was implemented in Python using a recursive descent parsing approach. The program was designed as a group of modular functions, where each function is responsible for a specific task.

3.1 Reading the Grammar

The first step is reading grammar rules from a file. Each line in the grammar file contains one production rule. The left-hand side and right-hand side of the rule are separated by the `::=` symbol. If there are multiple alternatives, they are separated by the vertical bar symbol `|`.

For example:

$$\langle \textit{verb-phrase} \rangle ::= \langle \textit{verb} \rangle \mid \langle \textit{verb} \rangle \ \langle \textit{noun-phrase} \rangle$$

These rules are stored in a dictionary structure. The left-hand side is used as the key, and the right-hand side alternatives are stored as lists. This structure provides direct access to production rules during parsing.

3.2 Non-terminal and Terminal Detection

In order to parse correctly, the program must distinguish between non-terminal and terminal symbols. A symbol is considered non-terminal if it exists as a key in the grammar dictionary. Otherwise, it is treated as a terminal symbol.

This approach allows the parser to work with grammars that use different naming styles, such as angle-bracket notation (`<sentence>`) or single-symbol notation (`S, A, B`).

3.3 Recursive Descent Parsing

The core of the project is the `parse_symbol` function, which recursively processes the input according to grammar rules.

In the implementation, the parsing process is controlled by a function called `parse_symbol`. This function takes the current symbol and input position as parameters and recursively tries to match the input according to grammar rules.

Each time a non-terminal is encountered, the function retrieves all possible productions and tries them one by one. If a match fails, the parser backtracks to the previous position and tries another alternative.

If the symbol is terminal, it is directly compared with the current token. If they match, parsing succeeds and the position moves forward by one token. If they do not match, parsing fails.

If the symbol is non-terminal, the parser obtains all alternatives of that symbol from the grammar and tries them one by one. This process is recursive because each non-terminal may expand into other non-terminals.

3.4 Backtracking

When a non-terminal has more than one alternative, the parser may need to try several possible productions. For this reason, backtracking is used. Before trying an alternative, the current token position is saved. If the alternative fails, the parser restores the previous position and tries the next alternative.

This is important for rules such as the following:

For example, when parsing a verb phrase, the parser may first try a shorter production such as `<verb>`. If the remaining input does not match, the parser backtracks and tries a longer production such as `<verb> <noun-phrase>`.

$$\langle \textit{verb-phrase} \rangle ::= \langle \textit{verb} \rangle \mid \langle \textit{verb} \rangle \ \langle \textit{noun-phrase} \rangle$$

In such cases, the longer alternative is tested first in order to reduce incomplete parsing situations.

3.5 Handling Epsilon Productions

Some grammars include epsilon productions, where a rule can derive the empty string:

$$A ::= aA \mid \epsilon$$

In this project, epsilon is not treated as an input token. Instead, it is handled as a special successful case that consumes no input. This design allows the parser to correctly process grammars that contain optional or repeated structures.

One limitation of this approach is that recursive descent parsing may not efficiently handle left-recursive grammars without modification. Future improvements could include transforming such grammars or implementing more advanced parsing techniques.

3.6 Tokenization Strategy

Two different grammar styles were tested in this project:

- Word-based grammar
- Character-based grammar

For word-based grammars, the input sentence is split by spaces. For character-based grammars, the input is divided character by character. This makes the parser flexible enough to support both types of grammar structures.

3.7 Parse Tree Construction

During parsing, the program also constructs a parse tree. Each successful parse step creates a node. Terminal nodes correspond to actual tokens, and non-terminal nodes correspond to grammar symbols that expand into substructures.

The tree is stored in dictionary-like nested structures and later printed in an indented format for readability.

3.8 JSON Conversion

After generating the parse tree, the parser converts it into JSON format. Terminal values are simplified to strings, while non-terminal values become nested key-value structures.

This conversion is important because JSON provides a structured and machine-readable representation of the parse tree, which can be easily used in further processing such as compilers, interpreters, or visualization tools.

For recursive chains such as:

$$A ::= aA \mid \epsilon$$

the JSON representation is compressed into a simpler and more readable form such as:

```
"A": "aaa"
```

instead of deeply nested recursive objects.

3.9 Error Handling

For invalid inputs, the parser does not only report that the sentence is incorrect. It also records the farthest position reached during parsing and the expected symbol or symbols at that location.

This allows the program to generate meaningful error messages in the following form:

- where the error occurs,
- what was expected,
- why the sentence is invalid.

4 Findings

The parser was tested with two different grammars.

4.1 Grammar 1: Word-Based Grammar

The first grammar was an English-like grammar using noun phrases and verb phrases.

4.1.1 Example Parse Tree Output

```
--------------------------------------------------
Input: the man saw a dog
Valid
Parse Tree:
<sentence>
  <noun-phrase>
    <determiner>
      the
    <noun>
      man
  <verb-phrase>
    <verb>
      saw
    <noun-phrase>
      <determiner>
        a
      <noun>
        dog
```

Figure 1: Parse tree generated for the input “the man saw a dog”

4.1.2 Example JSON Output

```
JSON:
{
  "sentence": {
    "noun-phrase": {
      "determiner": "the",
      "noun": "man"
    },
    "verb-phrase": {
      "verb": "saw",
      "noun-phrase": {
        "determiner": "a",
        "noun": "dog"
      }
    }
  }
}
--------------------------------------------------
```

Figure 2: JSON representation generated for the input “the man saw a dog”

4.1.3 Example Error Output

```
Input: liked the dog
Invalid
Error:
- Where the error occurs: token 1 ("liked")
- What was expected: a, the
- Why the sentence is invalid: sentence does not match the required starting structure
```

Figure 3: Error message for invalid input "liked the dog"

These outputs demonstrate that the parser can successfully generate both a hierarchical parse tree and a structured JSON representation for valid input sentences.

Examples of valid inputs:

- `the man saw a dog`
- `a telescope admired the cat`
- `a man liked`

Examples of invalid inputs:

- `the man the dog`
- `liked the dog`

For valid inputs, the parser produced correct parse trees and JSON outputs. For invalid inputs, the parser correctly reported the error location and expected symbols.

These results show that the parser can correctly identify sentence structures and produce hierarchical outputs. The error detection mechanism also helps users understand why an input is invalid.

4.2 Grammar 2: Character-Based Grammar

The second grammar used recursive productions:

```
S ::= A B
A ::= a A | \epsilon
B ::= b B | \epsilon
```

This grammar accepts strings of the form $a^n b^m$, where $n \geq 0$ and $m \geq 0$.

4.2.1 Example Parse Tree Output

```
Input: ab
Valid
Parse Tree:
S
  A
    a
    A
      ε
  B
    b
    B
      ε
```

Figure 4: Parse tree for input “aaaaaaaaaabbbbbbbbb”

4.2.2 Example JSON Output

```
JSON:
{
  "S": {
    "A": "a",
    "B": "b"
  }
}
```

Figure 5: Compressed JSON output for recursive structure

This example demonstrates the parser’s ability to handle recursive structures and epsilon productions. The input consists of repeated characters, and the parser successfully reduces them into a compact JSON representation.

4.2.3 Epsilon Example

```
Input: ε
Valid
Parse Tree:
S
  A
    ε
  B
    ε

JSON:
{
  "S": {
    "A": "ε",
    "B": "ε"
  }
}
```

Figure 6: Handling of epsilon (empty string) input

This example demonstrates how the parser handles epsilon (empty string) productions. Even when no input is provided, the parser correctly derives the empty structure using recursive rules.

This highlights the flexibility of the parser in handling both structured language inputs and abstract formal grammars.

Examples of valid inputs:

- `ab`
- `aaa`
- `aaaaaaaaaabbbbbbbbbb`
- ϵ

Examples of invalid inputs:

- `ba`
- `abab`

The parser successfully handled epsilon productions and generated compressed JSON outputs for recursive structures.

This demonstrates that the parser can successfully handle recursive definitions and epsilon productions, which are essential features in many formal grammars.

5 Difficulties Encountered During Implementation

During the implementation of the project, several difficulties were encountered. One of the main difficulties was designing the recursive descent parser in a way that could work with different grammar files instead of only working for a fixed example grammar. Since

the grammar rules are read from `grammar.txt`, the parser had to process non-terminal and terminal symbols dynamically.

Another challenge was implementing backtracking correctly. When a rule had more than one alternative, the parser needed to save the current token position before trying an alternative. If that alternative failed, the parser had to return to the previous position and try the next possible production. This was especially important for rules such as `<verb-phrase> ::= <verb> | <verb> <noun-phrase>`.

Handling epsilon productions was also challenging because epsilon represents the empty string and should not consume any input token. Therefore, epsilon had to be treated as a special successful case during parsing. This was necessary for recursive grammars such as `A ::= a A | \epsilon` and `B ::= b B | \epsilon`.

Another difficulty was generating meaningful error messages for invalid inputs. The program needed to keep track of the farthest parsing position reached and the expected symbols at that point. This made it possible to explain where the error occurred, what was expected, and why the sentence was invalid.

Finally, converting the parse tree into a clean JSON representation required additional work. Recursive structures can create deeply nested outputs, so the JSON formatting had to be organized carefully to make the result readable and understandable.

6 Conclusion and Discussion

In this project, a recursive descent parser was successfully implemented for grammars written in BNF format. The parser can read grammar rules from a file, process input sentences, detect whether they are valid, generate parse trees, and convert those trees into JSON representations.

This project provided hands-on experience in implementing a parser from scratch, which is a fundamental concept in compiler design and programming language theory.

One of the important strengths of the project is its flexibility. The same parser structure can handle both word-based and character-based grammars. In addition, epsilon productions are processed correctly, which is essential for many recursive grammars.

Another important result is the error reporting mechanism. Instead of simply rejecting invalid inputs, the system provides the location of the error and the expected symbols, making the output much more informative.

Overall, this project provides a practical example of syntax analysis and demonstrates how recursive descent parsing can be implemented for different grammar types.

7 Recommendations

The current project can be improved in several ways:

- More complex grammars could be tested.
- The parser could be extended to support ambiguity analysis.
- A graphical user interface could be added for easier interaction.
- JSON output formatting could be further customized for additional grammar styles.
- The parser could be extended into a larger compiler front-end project.

References

1. Course project description document.
2. BNF and recursive descent parsing lecture notes.
3. Python documentation.

Appendices

The source code, grammar files, and input sentence files are included as project attachments.